



SAPIENZA
UNIVERSITÀ DI ROMA

Rehabilitation effectiveness and efficiency inference

Faculty of Ingegneria dell' Informazione, Informatica e Statistica
Corso di Laurea Magistrale in in Computer Science

Candidate

Anxhelo Diko

ID number 1867167

A handwritten signature in blue ink that reads 'Anxhelo Diko' with a stylized flourish underneath.

Thesis Advisor

Prof. Luigi Cinque

A handwritten signature in blue ink, appearing to read 'Luigi Cinque'.

Co-Advisors

Prof. Danilo Avola

Prof. Massimiliano Magnone

Academic Year 2019/2020

Thesis not yet defended

Rehabilitation effectiveness and efficiency inference

Master's thesis. Sapienza – University of Rome

© 2020 Anxhelo Diko. All rights reserved

This thesis has been typeset by L^AT_EX and the Sapthesis class.

Author's email: angelo.diko12@gmail.com

"I have not failed. I have just found 10,000 that will not work"

-Thomas Edison

Dedicated to my family

Acknowledgments

First and foremost, I would like to express my gratitude to Professor Luigi Cinque who was close to me during my master degree together with professor Danilo Avola. Thanks for being such a wonderful persons and always being available when it mattered the most.

Secondly, a heartfelt thanks to the Sapienza VisionLab members for being always abilable to hepl, especially Alessio Fagioli who encouraged and helped me through every difficulty.

Thirdly, my gratefulness goes toward my family as well as my friends, who allowed me to reach my goals by supporting me through the years, especially when things were pretty difficult.

Contents

1	Introduction	1
1.1	Aims and scope	1
1.2	State of the art	2
1.3	Contribution and thesis outline	4
2	Data Preprocessing	1
2.1	Machine Learning workflow	1
2.2	Feature enhancement	2
2.3	Dimensionality reduction	4
2.3.1	Feature Extraction	5
2.3.2	Feature Selection	5
2.4	Conclusion	8
3	Machine Learning	1
3.1	Introduction	1
3.1.1	Learning Paradigms	2
3.2	Basic Machine Learning Algorithms	3
3.2.1	Linear Regression	3
3.3	Decision Trees	5
3.4	Ensemble Learning	6
3.5	Advanced Machine Learning algorithms	8
3.5.1	Multi-Layer Perceptron	8
3.5.2	Gradient Boosting Tree (GBT)	11
3.6	Conclusions	17
4	System Architecture	1
4.1	Method	1
4.2	Feature Preprocessing	2
4.3	Model Architecture	4
4.4	Conclusions	5
5	Test and evaluation	1
5.1	Dataset	1
5.2	Implementation details and model analysis	2
5.3	Comparison	3
5.4	Conclusions	5

6	Conclusions	1
---	-------------	---

Chapter 1

Introduction

This chapter is organized as follows. In section 1.1 the candidate's scope and aims are presented. In section 1.2 the state of the art in relation to the topic is summarized. Finally, in section 1.3 the contribution of the proposed work is highlighted and the structure of the thesis is also provided.

1.1 Aims and scope

One out of six people in the European Union has a disability , usually caused by an acute episode or a chronic disease[1]. Providing a suitable rehabilitation is one of the main concerns for people, especially as they age. A big importance is given to rehabilitation improvement in the recent years, trying to use information technology to exploit and also collect data about the patients that helps evaluating treatments and rehabilitation strategies.

Evaluating progress throughout a patient's rehabilitation episode is critical for determining effectiveness and efficiency of the selected treatments and contributing to the evidence-based practice. It is of key importance to assess in advance the potential functional improvement of patients undergoing a rehabilitation program because this may help in developing precision medicine tools and to assess a rehabilitative path that is patient specific[2]. To serve this major goal, it is necessary to have a Rehabilitation Measure of Outcome (RMO) that can be a qualitative or quantitative measurement of the outcome in response to rehabilitation intervention in the context of physiatry[3]. There are plenty of reasons why RMOs are important. They help tracking changes in functioning at an individual as well as population level. They establish a common language that can be understood by all rehab professionals, as well as insurance companies that influence health provision[3]. They help provide feedback for improving clinical rehabilitation interventions, as well as answering research specific questions.

The most studied and validated rehabilitation outcome measures used in Inpatient Rehabilitation Facilities are Functional Independence Measure (FIM) and Barthel Index (BI)[3]. In this work the focus will be on BI, it is an indicator that evaluates

Activities Daily Living (ADL) of a patient. The BI has a range from 0 to 100, which evaluates the patient's condition based on ten main daily life activities. Each of the main activities is scored with 5 potential values, where the higher the value the better the ability in performing the task with zero reflecting the total inability. In the current literature it is shown that BI has been the focus of many studies related to rehabilitation improvement. The most used approach is to apply statistical analysis on patients data for identifying what are the most important factors that affect the rehabilitation process[2] meanwhile we propose a new approach with respect to Barthel Index.

In this work, evaluating progress throughout a patient's rehabilitation in terms of effectiveness and efficiency is going to be explored by using advanced formal models comprising, among many others, patient BI before rehabilitation, intervention cause, related actual disease and patient medical history by taking full advantage of specific machine learning architectures. The rationale behind this study is due to the importance of assessing in advance the potential functional improvement of patients undergoing a rehabilitation program in terms of developing precision medicine tools and patient oriented rehabilitation programs, as well as to better allocate resources in hospital since there is an increasing pressure for performance measurement in this era of ballooning health care expenditure and growing emphasis on accountability[4]. Furthermore, the work is also motivated by the absence of research with respect to rehabilitation for predicting effectiveness and efficiency which is worth study because of the impact it can have in improving healthcare.

1.2 State of the art

Rehabilitation outcome prediction is a well established topic and there are a few researches in this field, which are able to reach satisfactory results in different tasks related to rehabilitation outcome. A rather straightforward approach to rehabilitation outcome measurement is to use clinical outcome measures to quantify a patient's impairment and function[5]. This approach requires the rehabilitation to be over and measures patient specific Activity Daily Living related indicators to come out with an evaluation regarding the rehabilitation outcome. The fact that requires rehabilitation to be over, makes this approach not suitable in any case.

The most common type of strategy, when it comes to evaluating rehabilitation, is to use statistical predicting tools that can be applied to the available patient data before the rehabilitation is over[6]. For this kind of approach there are indeed various works available in literature and they are able to reach considerable accuracy rates. Automatic systems that tackle the rehabilitation evaluation task usually follow a series of standard steps such as data preprocessing, feature selection, and classification or prediction depending on the form of the problem, explained as following:

- *Preprocessing*: This is the process where different transformations are applied to the data. They usually consist in scaling (if needed), transforming the distri-

bution to make it more similar to a normal distribution, encoding categorical variables, etc. The reason behind these actions on the data is to make them more suitable for the learning systems. These are standard steps for medical data[6].

- *Feature Selection*: Relevant features are selected to be part of the learning models. To select the features, a common understanding of the problem is required or the use of statistical tools. One example is the use of Random Forests that assigns an indicator to features ,called Importance Score (IS), with respect to the target variable and those with the higher IS are selected[5].
- *Classification/Prediction*: Based on the nature of the task, a model is fitted, usually a regression model or a decision tree. If it is a classification problem it classifies an instance based on the given data or if it is a continuous value it tries to predict it based on the given data relying on the fitted model[5].

Moving on with other available approaches, there are methods that use reinforcement learning for computer-assisted patient rehabilitation. In this approach the trained agent can stimulate the outcome that a certain action on the patient can have during rehabilitation, as each possible activity to a patient at a single point in time can have different outcomes and the system decides if it can be recommended or not. It is very useful for designing rehabilitation pathways related to the patient condition that are monitored in continuity[6].

A completely different approach is the use of Support Vector Machines (SVM) and K-Nearest Neighbours (KNN) to guide rehabilitation planning. In this approach SVM and KNN were used to support the decision making system regarding home rehabilitation for older people and was compared to a standard test in medicine known as Activity Daily Living Care Plan Responses (ADLCAP)[7]. It was shown that KNN and SVM could make better decisions than the standard test.

Some other works regarding rehabilitation outcome prediction and not only, have tried some interesting approaches that combine Deep Learning with traditional Machine Learning models. An example is the use of Deep Neural Network, usually Multilayer Perceptron, combined with a gradient boosting tree resulting in an ensemble method. This method has proven to be very accurate in rehabilitation decision making and many related tasks with small scientific datasets. It also provides the possibility of multi tasking resulting in an architecture called Boosting Tree-Assisted Multi Task Deep Learning[56].

The last interesting point, which also strongly motivated this work, is that basically all of the available research are classifiers that are used for decision making but none of them tries to predict outcome measures like Barthel Index. Assessing in advance the functional potential of a patient can be of key importance for designing rehabilitation pathways from the beginning. Being able to do that could have a big impact in designing precision medicine tools and provide a per-patient effective rehabilitation pathway since the beginning.

1.3 Contribution and thesis outline

In this work the problem of effectiveness and efficiency in rehabilitation will be explored by making use of a dataset provided from the "Policlinico Umberto I" that contains relevant information accumulated in years from a considerable number of patients. The novelty lies with the usage of new and more advanced machine learning models instead of being limited to traditional machine learning and statistical models like SVM or Feed Forward Neural Networks. To handle the task, the system will make use of advanced ensemble architectures such as the Gradient Boosting Machines and exploit advanced combination techniques like stacking to combine multiple models into a more powerful one. The main contributions of the proposed work, in relation to the current literature in effectiveness and efficiency in rehabilitation, can be summarized in the following key points:

- The proposal of a new classification system for diseases. The original classification system is too sparse and because of the small size dataset, the sparsity would not allow us to actually learn relevant informations so we proposed a new classification system on the given dataset known as ICD-9 (International Classification of Diseases, Ninth Revision).
- The introduction of new and more advanced machine learning techniques by leveraging models like xGBoost, lightGBM and catboost that achieves SOTA result in many tasks including also medical domain tasks. Stacking techniques were used as an ensemble method to combine the used models.

The rest of this thesis is structured as follows. Chapter 2, Feature Analysis, Introduces preprocessing techniques that were heavily used in this work to prepare the data for the machine learning models. Chapter 3, Machine Learning, presents the most common algorithms used in machine learning and also some advanced ones related to this work. Chapter 4, System Architecture, shows how the dataset was used to create the input for the system, and how the chosen architectures were built and fitted, motivating the various choices. Chapter 5, Test and Evaluation, discusses the results obtained from the implemented architecture by showing how the proposed solution can give better performance than other modes used in previous works from literature. Finally, Chapter 6, Conclusions, presents what are the thoughts about this work and how the proposed solution can be further improved in the future.

Chapter 2

Data Preprocessing

In Machine Learning everything starts with emphasizing a problem and every Machine Learning problem is related to some data. What it does is builds a mathematical model based on sample data known as training data in order to make predictions or decisions to solve a problem without being programmed to do so[9]. For semi-structured medical data, this part of work is usually the most important one, that's why this chapter is focused on data and is organized as follows. In section 2.1 describes where the data preprocessing stays in a typical machine learning workflow. In section 2.2 the process of feature enhancement (techniques used on this work are presented) and its importance in Machine Learning models is described. In section 2.3 dimensionality reduction techniques used to extract features from our dataset are described. In section 2.4 the conclusions are shown emphasizing the most relevant information of this chapter.

2.1 Machine Learning workflow

A Machine Learning project implementation starts with a dataset and finishes with a model. In between these two parts, the dataset (raw data) and the models stands the data preprocessing part. Often, data preprocessing is the most important phase of a machine learning project, especially in computational medicine[10]. It is normally composed of two main components, data cleaning and feature engineering as seen in Figure 2.1.

Data cleaning is the process of detecting and correcting (or removing) corrupt or inaccurate records from a data set and refers to identifying incomplete, incorrect, inaccurate or irrelevant parts of the data and then replacing, modifying, or deleting the dirty or coarse data[11]. Data cleaning should always be the first step in the model building pipeline to ensure that the algorithms work with (largely) accurate data. The corrupted instances of the data may be created from different reasons like an error in data insertion, an error in measurement, etc. but the main issue stays that they can be fatal to the machine learning model. The way to handle corrupted data depends on the kind of data we are dealing with and the importance of the

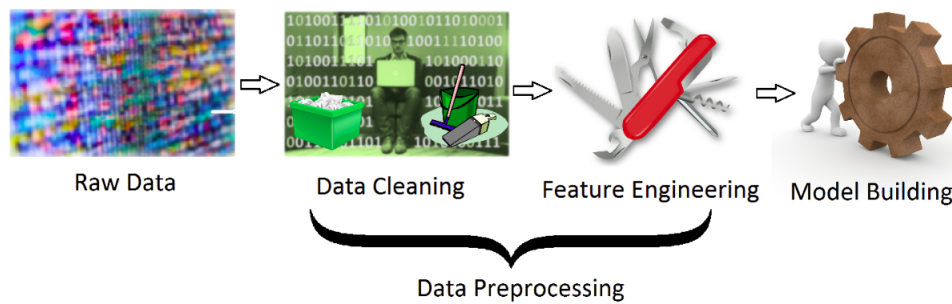


Figure 2.1. Machine learning workflow. Image taken from: <http://www.machinelearningtutorial.net/feature-engineering-in-machine-learning>

feature we are analysing but the final goal of data cleaning is the same in all cases, to build a consistent dataset.

Feature engineering involves designing manually what the input x 's should be, either using domain knowledge or exploiting statistical dependencies between features and outcomes.[12] In simple words it is the process of extracting features from a raw dataset. Without relevant features, you can't train an accurate model, no matter how complex the machine learning algorithm. Some of the most important techniques for feature engineering are feature enhancement, dimensionality reduction and feature selection.

2.2 Feature enhancement

Feature enhancement also called feature creation or transformation involves mapping a set of values for the feature to a new set of values (Figure 2.2) to make the representation of the data more suitable or easier to process for the downstream analysis. There are two main approaches for this task, one is taking information from the experts of the domain and transforming this information into features and the other approach is data driven. The data driven approach is where you try to add features that might not necessarily be something the domain experts would associate to the target, but practically it ends up being a good predictor anyway. That might happen when a feature really is an approximation for something else harder to observe directly but correlated to the target.

The feature transformation techniques differ from one type of data to another. For example if we consider categorical data the most common feature transformation techniques are: **introducing a popularity feature** (replace categorical features with high cardinality by their counts), **label-encoding**[13] (replacing categories with numbers, usually applied to ordinal data), **dummy encoding or one-hot encoding**[5] (creating binary features for $n-1$ categories that the data may have),

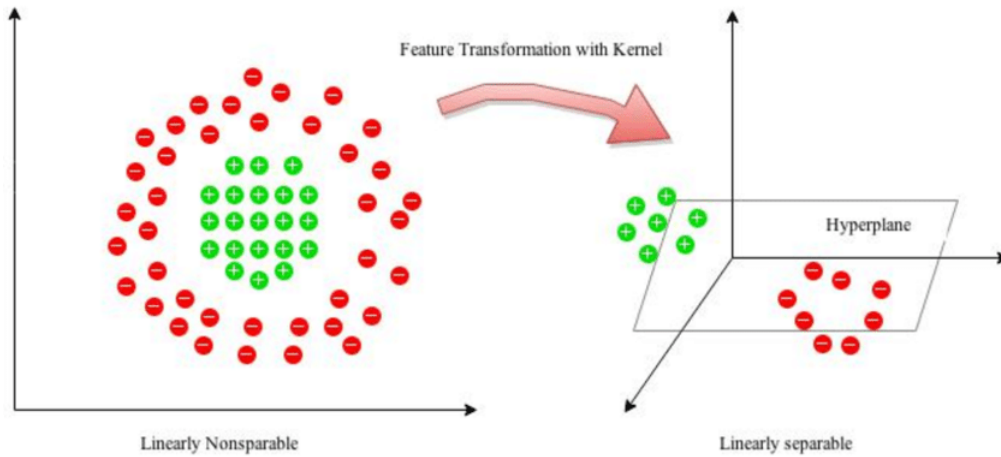


Figure 2.2. Feature Transformation using Kernels. Image taken from: https://www.researchgate.net/publication/280306676_Data-Based_Analysis_and_Learning

etc.. On the other side if we consider quantitative data some commonly used techniques are: **binning** (a technique that allows the representation of a continuous variable as a categorical, (Figure 2.3), **normalizing** (using the mean and standard deviation to create a shifted and scaled distribution like an approximation of a normal distribution), **using non-linear transformations** (for example adding the squared feature of a variable in linear regression), etc.. Following we are going to show some of the most common techniques used for feature transformation, feature scaling and dealing with skewed distributions.

Since the range of values of raw data varies widely, in some machine learning algorithms, objective functions will not work properly without normalization. For example features with higher range normally have a bigger impact than other features when predicting the target feature even though their discriminative power with respect to it might be lower. Therefore, the range of all features should be normalized so that each feature contributes approximately proportionately to the final distance. Feature scaling is a method used to normalize the range of independent variables or features of data. There exists different feature scaling techniques, the one we have tried during our work are the following:

- *Standardization*) (*Z-Score Normalization*): Feature standardization makes the values of each feature in the data have zero-mean (when subtracting the mean in the numerator) and unit-variance[22].

$$x' = \frac{x - \bar{x}}{\sigma} \quad (2.1)$$

- *Mean Normalization*

$$x' = \frac{x - \text{average}(x)}{\text{max}(x) - \text{min}(x)} \quad (2.2)$$

- *Rescaling (min-max normalization)*: Also known as min-max scaling or min-max normalization, is the simplest method and consists in rescaling the range of features to scale the range in $[0, 1]$ or $[-1, 1]$.

$$x' = \frac{x - \text{min}(x)}{\text{max}(x) - \text{min}(x)} \quad (2.3)$$

Often in our dataset we have continuous variables that are more useful as categorical variables. In this case what you want to do is discretise them in a smaller number of bins. The method that does this transformation is called data binning or data discretization. A good example of this technique is grouping of people based on their age (Figure 2.3). Binning can also speed up the learning algorithm[23].

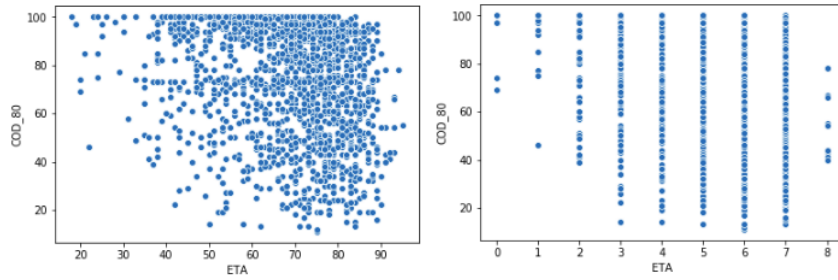


Figure 2.3. The relationship view between age and target variable before and after applying binning technique. Image generated from matplotlib.

2.3 Dimensionality reduction

Very often we face datasets that have a huge number of features and also often we find the need to reduce the number of features for faster training and sometimes better accuracy. In Machine Learning **dimensionality reduction** or **dimension reduction** is the process of reducing the number of random variables under some consideration[14]. Dimensionality reduction techniques have become an obvious need in the medical field (automated application). Today, a huge amount of data is generated in the medical domain. There exists two standard ways or methods for dimensionality reduction: **Feature Extraction** and **Feature Selection**(Figure 2.4).

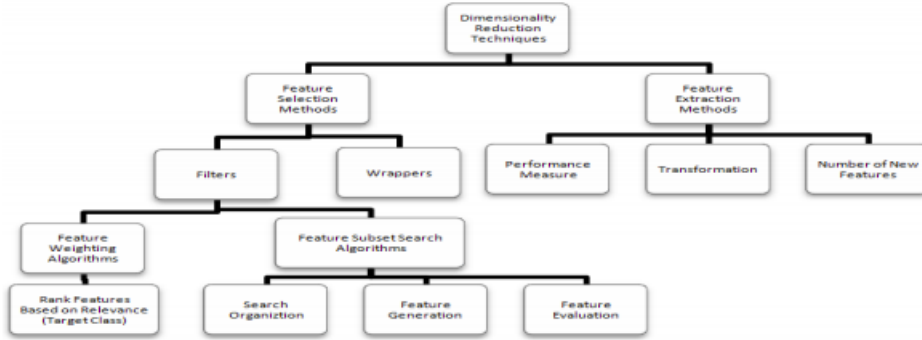


Figure 2.4. Dimensionality reduction techniques[19].

2.3.1 Feature Extraction

Feature Extraction is a general method in which a transformation of the input space onto the low-dimensional subspace that preserves most of the relevant information is made[21]. Generally feature extraction techniques are used to create a linear transformation of continuous features which have good discriminatory power. One of the most well known feature extraction techniques is called **Principal Component Analysis** or PCA. PCA is also one of the first techniques we used in this work to see how dimensionality reduction with feature transformation (linear transformation) would affect our model accuracy.

PCA is an unsupervised linear algorithm that creates a low dimensional representation of the original data(Figure 2.5) by describing as much variance as possible in the data[15]. It is a linear transformation of data that minimizes the redundancy (measured through covariance) and maximizes the information (measured through the variance). In simple words the PCA combines our input variables in a specific way, that allows to drop the "least important" variables while still retaining the most valuable parts of all of the variables. As an added benefit, each of the "new" variables after PCA are all independent of one another. This is a benefit because the assumptions of a linear model require the independent variables to be independent of one another especially in regression problems like the one we are facing in this work, where correlation between variables brings a phenomenon called multicollinearity that affects the accuracy of the model.

2.3.2 Feature Selection

High dimensional data may be problematic for different machine learning problems [16] including also medical related problems like our case. Beside the feature extraction technique (dimensionality reduction or data transformation) there is another well known technique called **feature selection**. Feature selection or feature subset selection is the process of selecting best features among all the features that are useful to discriminate a target feature. With feature selection no information

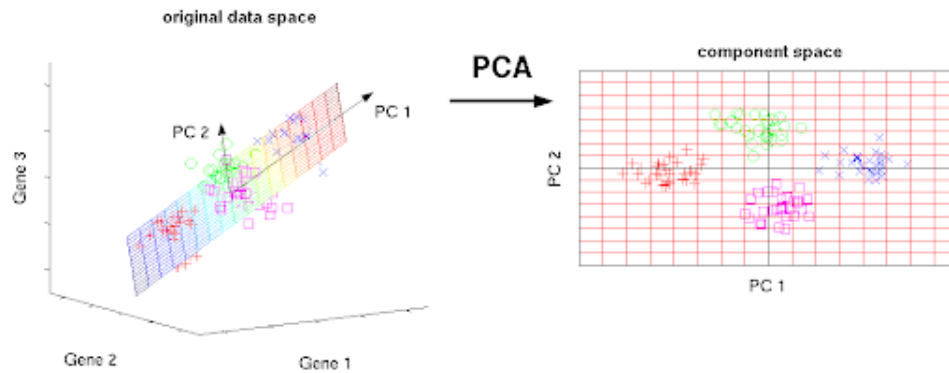


Figure 2.5. Feature extraction with principal component analysis (PCA). Image taken from: <https://medium.com/analytics-vidhya/math-intuition-and-implementation-of-principal-component-analysis>.

about the importance of a single feature is lost but if a small set of features is required and original features are very diverse, information can be lost as some of the features must be omitted during the feature subset selection process. Feature selection methods can be distinguished into three categories: wrappers, filters, and embedded/hybrid method. All of the mentioned methods were used on this work for testing purposes aiming to find the best subset that would give the best accuracy in the evaluation phase.

Wrapper methods work by evaluating a subset of features using a machine learning algorithm that employs a search strategy to look through the space of possible feature subsets, evaluating each subset based on the quality of the performance of a given algorithm[19]. These methods are called greedy algorithms because they aim to find the best possible combination of features that result in the best performing model which will be computationally expensive, and often impractical in the case of exhaustive search, even though it was our main dimensionality reduction techniques used to select the best feature subset we could. We used this method on this work and repeated the following standard workflow hundreds of times until we reached the best result. The standard workflow is:

- *Search for a subset of features:* Using a search method, select a subset of features from the available ones.
- *Build a machine learning model:* In this step, a chosen ML algorithm is trained on the previously-selected subset of features.
- *Evaluate model performance:* The newly-trained ML model is evaluated with a chosen metric.

- *Repeat*: The whole process starts again with a new subset of features, a new ML model trained, and so on.

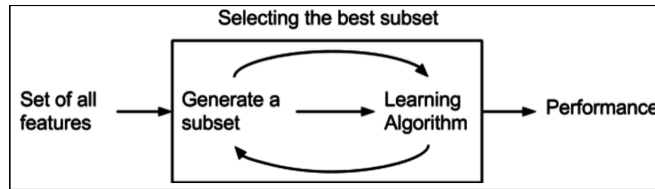


Figure 2.6. Feature selection using wrapper method[19].

The algorithm stops when a criteria is met and the subset with the best evaluation result is chosen. Another important part of the wrapper methods are the searching methods. The four searching methods used in this technique are as follow:

- *Forward Feature Selection*: This method starts with no feature and adds one at a time.
- *Backward Feature Elimination*: This method starts with all features present and removes one feature at the time.
- *Exhaustive Feature Selection*: This method tries all possible feature combinations.
- *Bidirectional Search*: And this last one does both forward and backward feature selection simultaneously in order to get one unique solution.

Filter methods select variables regardless of the model(Figure 2.5). They are based only on general features like the correlation with the variable to predict. Filter methods suppress the least interesting variables. The other variables will be part of a classification or a regression model used to classify or to predict data. These methods are particularly effective in computation time and robust to overfitting[9]. Filter methods tend to select redundant variables when they do not consider the relationships between variables. However, more elaborate features try to minimize this problem by removing variables highly correlated to each other[18].

Wrappers methods perform better than filter methods because the feature selection process is optimized for the classifier to be used. However, wrapper methods are expensive to be used for large feature space because of high computational cost and each feature set must be evaluated with the trained classifier that ultimately makes the feature selection process slow. Filter methods have low computational



Figure 2.7. Feature selection using filter method[19].

cost and faster but with inefficient reliability in classification as compared to wrapper methods and better suitable for high dimensional data sets.[19] Even though this method has benefits with respect to filter methods, we did not rely on it that much during our feature selection process.

Embedded methods utilize advantages of both filters and wrappers approaches[20]. A hybrid approach uses both an independent test and performance evaluation function of the feature subset [19]. In embedded techniques, the feature selection algorithm is integrated as part of the learning algorithm. In our case we applied this method by training different ensemble algorithms like Extreme Gradient Boosting Tree, Catboost, etc. and retrieving the feature importance out of them after the training phase. This methods follow a standard structure as follow:

- A machine learning model is trained
- Feature importance is derived from the trained model.
- The least important features are removed.

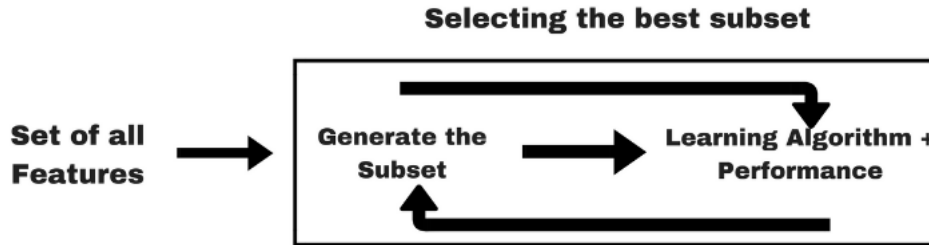


Figure 2.8. Feature selection using filter method[19].

The choice between feature extraction and feature selection methods depends on the specific data type and the domain of application. In medical data like our case, a good application of these techniques can be even more important than the model used itself.

2.4 Conclusion

In this Chapter the role of preprocessing in machine learning workflow was described and also some of the preprocessing techniques used during this work were presented

divided in two main categories, feature enhancement and dimensionality reduction (feature extraction and feature selection). The objective was to emphasize the role and the importance of data preprocessing on a machine learning problem and how crucial they are when you deal with a medical dataset like in our case. In problems like this one, data preprocessing is the most important part. A good application of the techniques we discussed in feature enhancement and dimensionality reduction may give a big boost to the machine learning algorithm. At the end of the day, some machine learning projects succeed and some fail, what makes the difference are the features used.

Chapter 3

Machine Learning

This chapter is focused on Machine Learning, it introduces the main algorithms used in this work and it is structured as follows. In Section 3.1 a general introduction of Machine Learning. Section 3.2 Introduces the key Learning paradigms. In Section 3.3 some of the most basic algorithms of Machine Learning are introduced. In Section 3.4 some advanced machine learning architecture or models are described. Finally in Section 3.5 a review of the treated topics is done.

3.1 Introduction

Machine Learning is a multidisciplinary field that studies algorithms that automatically improves from experience. It is often considered as a subfield of artificial intelligence that builds mathematical models based on sample data (experience), known as training data, in order to make predictions without being programmed to do so[9]. It is being used more and more in many application domains such as: computer vision [24][25], natural language processing[26], email filtering and also in medical domains from images to structured data. The goal of machine learning generally is to understand the structure of data and fit that data into models that can generalize, so the key concept is learning. The way machine learning models learn may be different from model to model and is also task dependent. It follows a certain flow that usually starts with an input (usually task related data) and ends with task execution and evaluation(Figure 3.1). In between these two phases the mode does what is called learning or training.

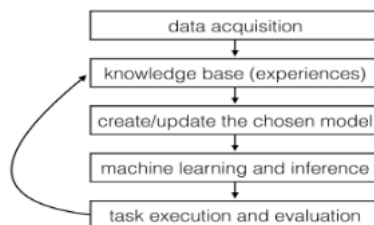


Figure 3.1. Machine learning generic model.

3.1.1 Learning Paradigms

In order to use the machine learning algorithms it is very crucial the training phase or the learning phase that was mentioned before, to be able to update it's weights and generalize well. Also as we stated the way models learn can be different. These different ways of learning often are named as learning paradigms. Some of the different proposed paradigms are explained in the following paragraphs.

Supervised Learning is the machine learning paradigm where the algorithm tries to map an input to an output based on input-output example pairs. So suppose we have a training set N , the training examples are of the form $(X_1, Y_1), \dots, (X_n, Y_n)$ where X_i represents the feature vector and Y_i its label that can be a class (classification problem), a continuous number (regression problem), etc.. It infers a function from the labeled training data $g : X \Rightarrow Y$ where X represents the input space and Y the output space. The required steps for this paradigm comprise a dataset generation to be used at training time, built by a field expert. A data preprocessing step, consisting in algorithm training in order to learn the given task (the g function). Also a validation step is needed to test the model accuracy and based on the result satisfactory the model can be retrained with different changes applied on it's hyperparameters.

Unsupervised Learning is the machine learning paradigm where the algorithm tries to detect previously undetected patterns in a data set with no pre-existing labels and with a minimum of human supervision. In contrast to supervised learning that usually makes use of human-labeled data, unsupervised learning, also known as self-organization allows for modeling of probability densities over inputs[27]. The main objective for this learning paradigm, is to find out the correct outputs by exploring and organizing data by using common characteristics among the provided samples, in order to aggregate them in the right way. Also here we have the data generation as a required phase, the training phase and also the validation phase to check the quality of the information clustering. Based on the results the model can again be reconfigured in case further training is needed.

Reinforcement Learning is a learning paradigm concerned with learning to control a system so as to maximize a numerical performance measure that expresses a long-term objective. What distinguishes reinforcement learning from supervised learning is that only partial feedback is given to the learner about the learner's predictions. Further, the predictions may have long term effects through influencing the future state of the controlled system. Thus, time plays a special role. The goal in reinforcement learning is to develop efficient learning algorithms, as well as to understand the algorithms' merits and limitations[28]. The usual form of reinforcement learning consists of an agent and an environment. The agent is a software that takes action on the environment (typically stated in the form of Markov Decision process[29]) with the main focus of achieving a balance between exploration and exploitation[30].

3.2 Basic Machine Learning Algorithms

In this section the fundamental machine learning and regression concepts related to our work will be presented. Even though they may seem not trivial, it is really important to have a good understanding of them as they are the building block of the advanced algorithms used in this work.

3.2.1 Linear Regression

Linear Regression is a statistical model that tries to explain a linear relationship between a scalar response (dependent variable) and one or more explanatory variables (independent variables). There are two general cases of linear regression, one in which there is only one independent variable and one when there are multiple independent variables. The first one is what is known as simple linear regression (Figure 3.2) and the later one is multiple linear regression [31]. Linear regression plays a fundamental role in the field of machine learning, it is one of the fundamental algorithms due to its simplicity and its characteristics. Mathematically speaking a linear regression can be described as follows: Given a dataset $Y_i, X_{i1}, \dots, X_{ip}$ for $i \in [1, n]$ a linear regression model assumes that the relationship between the dependent variable y and the p -vector of regressors x is linear. This relationship is modeled through a disturbance term or error variable ϵ an unobserved random variable that adds "noise" to the linear relationship between the dependent variable and regressors. Thus the model takes the form:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip} + \epsilon_i = \mathbf{X}_i^T \beta + \epsilon_i \quad (3.1)$$

Mean Squared Error (MSE)

From the above section we know that Linear Regression outputs an estimated value of the dependent variable y_i . To evaluate the accuracy of this estimator we need a function that measures the error rate between the predicted or estimated value and the ground truth or the real value. The mean squared error (MSE) of an estimator measures the average of the squares of the errors that is, the average squared difference between the estimated values and the actual value. MSE is a loss function, corresponding to the expected value of the squared error loss [32]. Mathematically speaking if we have a vector of n samples and Y is the vector representing the ground truth and $\hat{Y}$ is the estimated vector then MSE is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad (3.2)$$

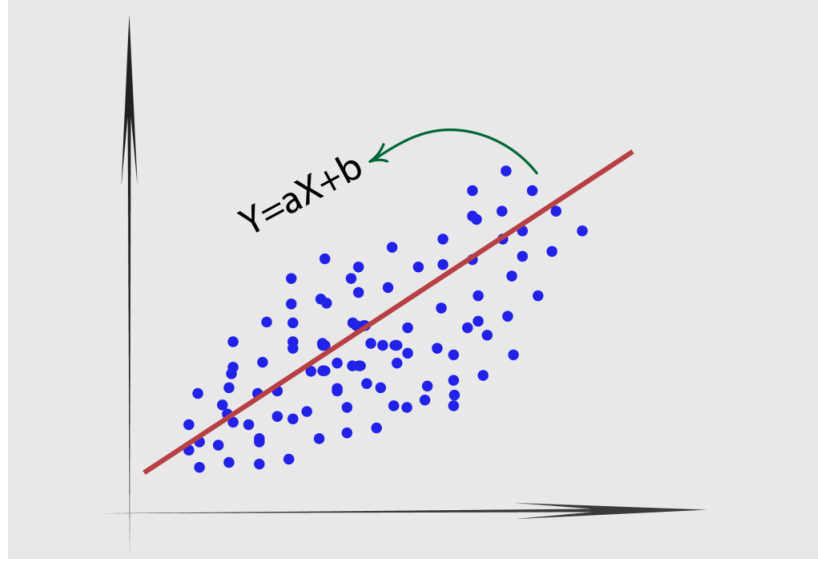


Figure 3.2. Simple linear regression. Image taken from: <https://www.machinelearning-mindset.com/linear-regression-with-tensorflow>

Regularizers Lasso, Ridge and ElasticNet

In terms of Machine Learning algorithm, linear regression tries to learn the β coefficients from the above formula (3.1) in order to generalize as well as possible. To improve generalization, in machine learning is often used a technique called regularization. Regularization is an important concept that helps with overfitting problems[33]. Overfitting happens when a model performs very well on a set of data (usually training data) but performs poorly on other sets of data, in simple words does not generalize. What regularization does is adding a penalty to the loss function that regularizes or shrinks the coefficient estimates towards zero. In other words, this technique discourages learning a more complex or flexible model, so as to avoid the risk of overfitting. As a loss function we saw above the MSE, to simplify things if we multiply MSE by n we take a measure called Residual Sum of Squares 3.3.

$$RSS = \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 \quad (3.3)$$

In statistics and machine learning, lasso (least absolute shrinkage and selection operator also Lasso or LASSO) is a regression analysis method that performs both feature selection and regularization in order to enhance the prediction accuracy and interpretability of the statistical model it produces[26]. Its formula is:

$$L_{lasso} = \sum_{i=1}^n (y_i - x_i' \beta_0)^2 + \lambda \sum_{j=1}^n |\hat{\beta}_j| \quad (3.4)$$

So from the formula we can easily say that it is RSS + a penalty of $|\beta_j|$. In statistics and machine Learning this technique is known as the L1 norm. So it applies the L1 norm to regularization for further improvement of the prediction. Another important regression regularization method is Ridge Regression (also known as weight decay) that can be obtained from lasso by replacing $|\beta_j|$ from the above formula with its square[34].

Very often is used a combination of these two regularizations called ElasticNet. ElasticNet regression is a combination of Ridge regression and lasso[36]. It is especially good at dealing with situations when there are correlations between parameters. This is because Lasso Regression tends to pick just one of the correlated terms and eliminates the others, whereas Ridge Regression tends to shrink all of the parameters for the correlated variables together. By combining these two ElasticNet regression groups and shrinks the parameters associated with the correlated variables and leaves them in equation or removes them all at once. This is possible by combining the penalties of Ridge Regression and lasso to get the best of both. ElasticNet aims at minimizing the following loss function:

$$E_{enet}(\hat{\beta}) = \frac{\sum_{i=1}^n (y_i - x_i' \hat{\beta})^2}{2n} + \lambda \left(\frac{1-\alpha}{2} \sum_{j=1}^m \hat{\beta}_j^2 + \alpha \sum_{j=1}^m |\hat{\beta}_j| \right) \quad (3.5)$$

where α is the mixing parameter between ridge ($\alpha = 0$) and lasso ($\alpha = 1$).

3.3 Decision Trees

Decision Tree algorithm is a tree based prediction algorithm used widely in data mining and machine learning[37]. It uses a decision tree approach to produce an output or make a prediction from an observation. There are two main types of decision trees: Classification trees when it has to predict a label and regression trees when the prediction can be any continuous value from a set of possible values. Trees in decision trees algorithms are implemented upside down starting from the root node (contains the data observations) and ending with leaf nodes where each leaf node represents a possible output (classification case, Figure 3.3). Nodes between root and leaf nodes are called internal nodes. In a decision tree, each internal node splits the instance space into two or more sub-spaces according to a certain discrete function of the input attributes values[37].

The tree in decision tree algorithms is built by splitting the data observations (coming as data input at the root node) into subsets of data which constitute the successor children nodes[38]. The splitting is based on a set of splitting rules based on classification features. It is a recursive process that is completed when the subset at a node has all the same values as the target. This kind of upside down or top

down induction is called a greedy algorithm and is by far the most used approach for decision tree models[39].

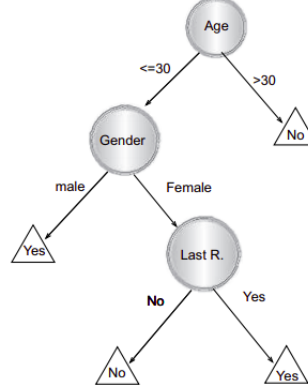


Figure 3.3. Decision tree example[37].

In decision tree algorithms metrics play an important role, they usually measure the homogeneity of the target within the subsets. One of the most used metrics for decision tree models is information gain. Information gain is an impurity-based criterion that uses the entropy measure (origin from information theory) as the impurity measure[37](3.7). Entropy on the other side is a measure that defines the homogeneity of a sample(3.6). For example when a sample is equally divided the entropy is 1 and when a sample is totally homogeneous the entropy is zero.

$$Entropy(y, S) = \sum_{c_j \in dom(y)} -\frac{|\sigma_{y=c_j} S|}{|S|} * \log_2 \frac{|\sigma_{y=c_j} S|}{|S|} \quad (3.6)$$

Information gain on the other hand measures the reduction in entropy by splitting a dataset according to a given value of a random variable.

$$InformationGain(a_i, S) = Entropy(y, S) - \sum_{v_i, j \in dom(a_i)} -\frac{|\sigma_{a_i=v_i, j} S|}{|S|} * Entropy(y, \alpha_{a_i=v_i, j} S) \quad (3.7)$$

3.4 Ensemble Learning

Ensemble Learning is a machine learning and statistical algorithm where different weak learners(models that perform poorly) are combined together(Figure 3.4) to

form a strong model that has a good accuracy and a good generalization[41 - 43]. Ensemble is a very popular technique and most of the top performing machine learning algorithms in different tasks use ensemble learning. Ensemble in itself is a supervised learning algorithm because it can be trained and then it can be used to make predictions. There are many ensemble algorithms but some of the most popular are bagging, boosting and stacking.

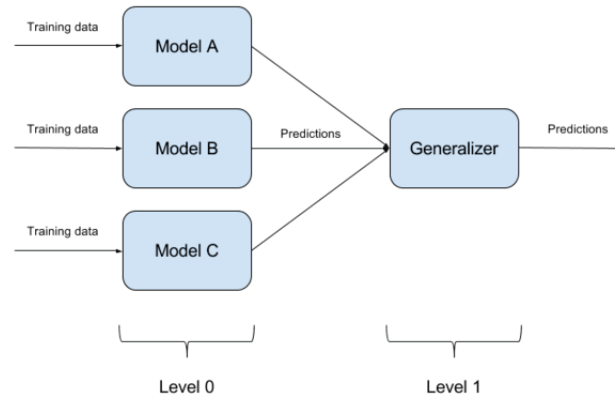


Figure 3.4. Ensemble Learning, combining estimators A,B and C through a generalizer to make predictions. Image taken from <https://www.kdnuggets.com/2020/01/random-forest-powerful-ensemble-learning-algorithm.html>

Bootstrap Aggregation (Bagging) is a machine learning ensemble meta-algorithm designed to improve the stability and accuracy of machine learning algorithms used in statistical classification and regression. The weak learners, in case of Bootstrap Aggregation are trained with different subsamples of the given data (bootstrap) and then they are aggregated or combined to produce a stronger model. The main benefit of Bagging is that it helps in avoiding overfitting and reduces the variance error.

Boosting represents a group of algorithms that utilize weighted averages to make weak learners into stronger learners. Unlike bagging that had each model run independently and then aggregate the outputs at the end without preference to any model. Boosting is all about "teamwork". Each model that runs, dictates what features the next model will focus on. In the following section we are going to see some of the best boosting ensemble algorithms.

Stacking (sometimes called stacked generalization) involves training a learning algorithm to combine the predictions of several other learning algorithms[45]. First, all of the other algorithms are trained using the available data, then a combiner algorithm, called meta-learner is trained to make a final prediction using all the predictions of the other algorithms as additional inputs. A property of stacking that distinguishes it from the other ensemble learning techniques, is the fact that in stacking we can have heterogeneous weak learners. The goal of stacking is to

produce a stronger less biased model that can achieve good generalization.

3.5 Advanced Machine Learning algorithms

In this section some advanced machine learning algorithms used for regression problems are presented. During our work we relied on these models to try to solve the problem of efficiency and effectiveness of surgery.

3.5.1 Multi-Layer Perceptron

A multi-layer perceptron is a machine learning algorithm used to discover non linear relationships between data[9]. It has a layered architecture where each layer is composed of smaller units called perceptrons. A perceptron is the simplest machine learning algorithm normally used for binary classification[40]. The way how perceptrons work is similar to a real neuron found on the human brain. A real neuron is composed of four main components, namely the dendrites, soma, axon, and synapses, which allow it to receive, elaborate as well as transmit electrical signals(Figure 3.5). Both synapses and dendrites ensure that these signals are transmitted and received among neurons. Somehow this is the functionality that an artificial neuron employs.

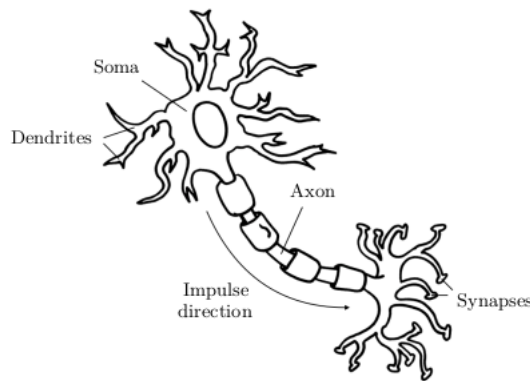


Figure 3.5. Neuron diagram showing machine learning formalized components.

The perceptron consists of four main parts including input values, weights and bias, net sum, and an activation function. The perceptron takes multiple inputs X_i and multiplies them by their assigned weights. After that, all the multiplied values are added together and create the weighted sum. To the weighted sum is also added a small value called bias and then it is applied to the activation function, producing the perceptron's output. The activation function plays the integral role of ensuring the output is mapped between required values. It is important to note that the weight of an input is indicative of the strength of a node. Similarly, an input's

bias value gives the ability to shift the activation function curve up or down. As an illustrative example we can see Figure 3.5 where a perceptron implementation is shown. The net_j represents the weighted sum plus bias W_j0 . Then net j is applied to the activation function to produce the output y_j .

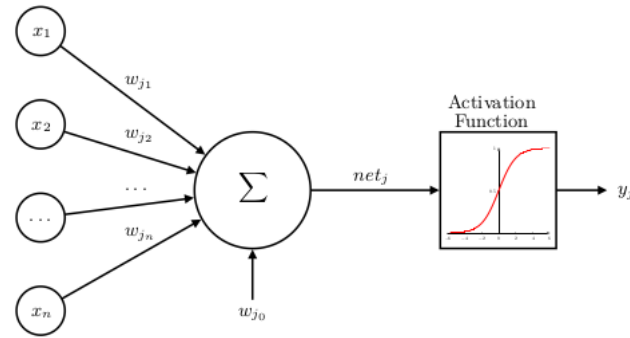


Figure 3.6. Perceptron implementation. A weighted sum of the inputs, net j is computed, then a sigmoid activation function is applied in order to decide the output y_j .

Now going back to multi-layer perceptron, we can say that it is just a layered collection of perceptrons (as stated before). Their structure consists of an input layer for receiving the signal (data), an output layer that makes a decision or prediction about the data and in between these two arbitrary layers are hidden layers that are the real engine of MLP (Figure 3.7).

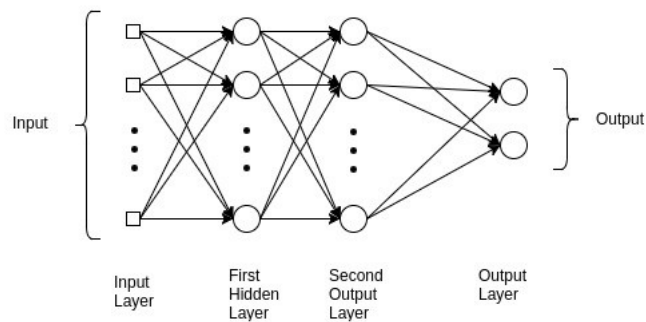


Figure 3.7. Multi-Layer Perceptron architecture. Image taken from https://medium.com/@AI_with_Kain/understanding-of-multilayer-perceptron-mlp

The most used form of multi-layer perceptron is called feed forward network. In this kind of architecture the information moves only in one direction from the input layer reaching the output layer and also all neurons or perceptrons are connected with neurons found on the next layer (also previous layer but the information does not move in that direction). Until now we have presented an architecture that learns

only linear relations, the algorithm that adds non linearity to the network is called backpropagation. With this technique, as said, an optimization function is used in order to choose the best weights among the various layers.

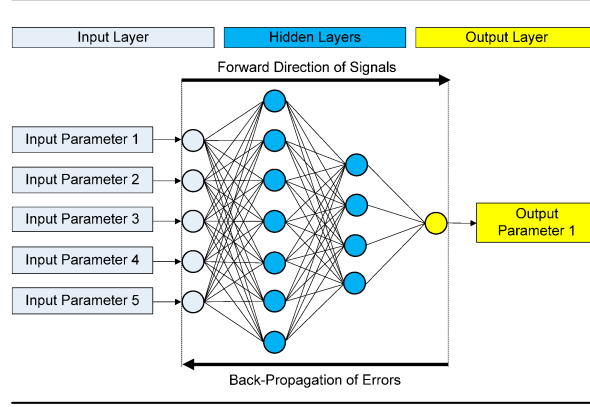


Figure 3.8. Feed Forward Neural Network and Backpropagation of error[56].

The optimization function tries for example to reduce the Mean Squared Error (MSE) between the network outputs and the real results(regression case), which are provided together with the inputs. The optimization function has to reduce, ultimately, the error value obtained from the following equation:

$$E = \sum_{s=1}^S \sum_{j=1}^n \frac{1}{2} (o_{sj} - y_{sj})^2 \quad (3.8)$$

where E is the value to be optimised, S is a non-empty training set, j indicates the j -th output unit, while o_{sj} and y_{sj} represent the network output and expected value for the j -th output unit, respectively. As stated above MLP has three different layers but only two of them need to update their weights and because there is no direct comparison with an output value for the hidden layers (supervised paradigm) MLP employs two different updating rules for each of the layers. The general rule, called delta rule is as follows:

$$\Delta w_{jk} = -\eta \delta_j o_k \quad (3.9)$$

where Δw_{jk} is the weight variation between neurons k and j , η indicates the learning rate, o_k represents the output of the k -th neuron and δ_j is the error computed for neurons belonging to either the output or hidden layers. A delta for a unit of the output layer is computed using the following equation:

$$\delta = y_j(1 - y_j)(o_j - y_j) \quad (3.10)$$

where, once again, o_j and y_j are, respectively, the predicted and real value of the j -th neuron. Regarding units in the hidden layer, the delta value computation is the following one:

$$\delta = y_j(1 - y_j) \sum_k \delta_k w_{kj} \quad (3.11)$$

where $\sum_k \delta_k w_{kj}$ is the weighted sum of errors produced by all units connected to the j -th neuron. Finally, the backpropagation algorithm updates weights using the following equation:

$$w_{kj} = w_{kj} + \Delta w_{jk} \quad (3.12)$$

3.5.2 Gradient Boosting Tree (GBT)

Gradient boosting is a machine learning technique for regression and classification problems, which produces a prediction model in the form of an ensemble of weak prediction models. It is normally used with decision trees (mostly CART) hence the name Gradient Boosting Tree (GBT)[46]. For a given data set with n examples and m features $D = (x_i, y_i) (|D| = n, x_i \in R_m, y_i \in R)$, a tree ensemble model uses K additive functions to predict the output (Figure 3.9). The prediction formula is:

$$\hat{y}_j = \phi(\mathbf{x}_i) = \sum_{k=1}^K f_k(\mathbf{x}_i), f_k \in F \quad (3.13)$$

where $F = f(x) = wq(x) (q : R_m \Rightarrow T, w \in R, T)$. The q represents the structure of each tree that maps an example to the corresponding leaf index and each f_k corresponds to an independent tree structure q and leaf weight w . T is the number of leaves in the tree.

For learning the set of functions GBT minimizes the following regularized function:

$$\Gamma(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k) \quad (3.14)$$

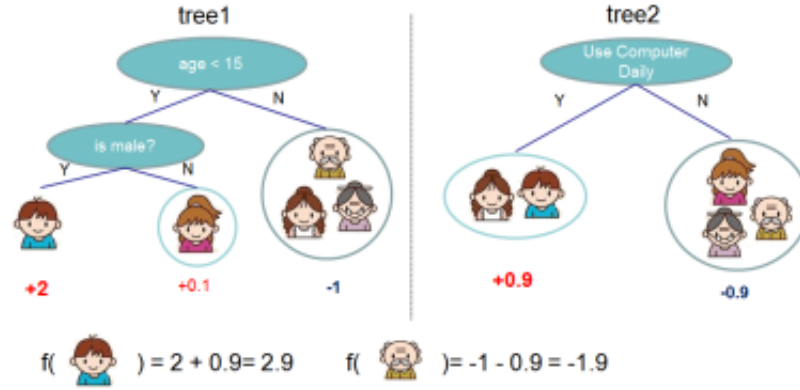


Figure 3.9. Gradient Boosting tree (ensemble tree) final prediction[46].

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (3.15)$$

Where l is a differentiable convex function that measures the difference between the predicted value and the ground truth. The second term Ω applies some penalty (regularization) to the model. Doing so requires a learning or training phase, in GBT the training is an additive greedy process. Formally during the learning process at t -th iteration we add f_t to (3.15) like following in (3.16) and try to minimize it[46].

$$\Gamma_{(t)} = \sum_i l(\hat{y}_i + f_t(\mathbf{x}_i), y_i) + \Omega(f_k) \quad (3.16)$$

So basically a f_t is added to the model greedily that most improve the model according to equation (3.16). To make the optimization process faster, a second order approximation term is added and we obtain the following formula:

$$\bar{\Gamma}_{(t)} = \sum_i l[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \Omega(f_k) \quad (3.17)$$

Where $g_i = \sigma \hat{y}_{(t-1)} l(y_i, \hat{y}_{(t-1)})$, $h_i = \sigma \hat{y}_{(t-1)}^2 l(y_i, \hat{y}_{(t-1)})$ are first and second order gradient statistics on the loss function. Note that equation (3.17) is derived from the original function by removing the constants. Now if we define $I_j = \{i | q(\mathbf{x}_i) = j\}$ as the instance set of leaf j , equation (3.17) can be written as follow by extending the $\hat{y}_{(t-1)}$:

$$\bar{\Gamma}_{(t)} = \sum_i l[g_i f_t(\mathbf{x}_i) + \frac{1}{2} h_i f_t^2(\mathbf{x}_i)] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (3.18)$$

Now for a fixed structure $q(x)$ the optimal value w_j^* of leaf j can be computed as follow:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \quad (3.19)$$

and calculate the corresponding optimal value by:

$$\bar{\Gamma}_{(t)} = -\frac{1}{2} \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \quad (3.20)$$

And now with (3.20) we have the scoring function of GBT that plays the same role as impurity function on the decision tree with the difference that is derived from a wider range of objective functions. Another important thing about GBT is that it uses a greedy algorithm that iteratively adds branches to the tree starting from a single leaf [46]. This because it is not possible to enumerate all possible tree structures q . Saying this, if after the split we have I_L and I_R for the left side split and right side split where $I_R \cap I_L = I$ we have the following final function:

$$\Gamma_{split} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)}{\sum_{i \in I_L} h_i + \lambda} - \frac{(\sum_{i \in I_R} g_i)}{\sum_{i \in I_R} h_i + \lambda} + \frac{(\sum_{i \in I} g_i)}{\sum_{i \in I} h_i + \lambda} \right] + \gamma T \quad (3.21)$$

Extreme Gradient Boosting Tree (xGBT)

xGBT is an improved implementation of GBT. This algorithm for many years now has proven to produce state of the art results in many tasks and is one of the most widely used in data science and machine learning problems. Mathematically speaking xGBT is almost the same as GBT, the main difference between xGBT and GBT stays on split finding algorithms for finding the perfect splits to optimizer function (3.21)[48]. xGBT offers many splitting algorithms that can help achieve good results in different conditions. The first splitting algorithm is Basic Extract Greedy Algorithm(Figure 3.10).

Algorithm 1: Exact Greedy Algorithm for Split Finding

Input: I , instance set of current node
Input: d , feature dimension
 $gain \leftarrow 0$
 $G \leftarrow \sum_{i \in I} g_i, H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ **to** m **do**
 $G_L \leftarrow 0, H_L \leftarrow 0$
 for j **in** $sorted(I, \text{by } \mathbf{x}_{jk})$ **do**
 $G_L \leftarrow G_L + g_j, H_L \leftarrow H_L + h_j$
 $G_R \leftarrow G - G_L, H_R \leftarrow H - H_L$
 $score \leftarrow \max(score, \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda})$
 end
end
Output: Split with max score

Figure 3.10. Exact Greedy Algorithm for Split Finding[48].

Exact Greedy Algorithm enumerates over all the possible splits on all the features. It is computationally demanding to enumerate all the possible splits for continuous features. In order to do so efficiently, the algorithm must first sort the data according to feature values and visit the data in sorted order to accumulate the gradient statistics for the structure score in equation (3.21). The next splitting algorithm used is called Approximate Algorithm (Figure 3.11)[48].

Algorithm 2: Approximate Algorithm for Split Finding

for $k = 1$ **to** m **do**
 Propose $S_k = \{s_{k1}, s_{k2}, \dots, s_{kl}\}$ by percentiles on feature k .
 Proposal can be done per tree (global), or per split(local).
end
for $k = 1$ **to** m **do**
 $G_{kv} \leftarrow \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} g_j$
 $H_{kv} \leftarrow \sum_{j \in \{j | s_{k,v} \geq \mathbf{x}_{jk} > s_{k,v-1}\}} h_j$
end
 Follow same step as in previous section to find max
 score only among proposed splits.

Figure 3.11. Approximate Algorithm for split finding[48].

The Approximate algorithm tries to address some issues regarding the Basic Exact Greedy algorithm. In terms of accuracy there is no way that Exact Greedy algorithms can be improved but it is not possible to use it with a big amount of data that can not fit entirely in memory and also with distributed data[48]. What Approximate algorithm basically does is that it first proposes candidate splits according to percentiles of feature distribution (based on some criteria). Then it maps the continuous features into buckets split by these candidate points,

aggregates the statistics and finds the best solution among proposals based on the aggregated statistics. There are also other binning techniques instead of quantiles but Approximation based on quantiles have proven to give accuracy very close and sometimes identical to Basic Exact Greedy.

The last split finding algorithm is called Sparsity-aware split finding algorithm (Figure 3.12). As the name suggests it is useful in cases where data is sparse. Sparsity in data can be caused from different factors like missing data, frequent zero entries in the statistics, etc. This is one of the key aspects of xGBT that makes it one of the best tree algorithms in different tasks, handling sparse data. The key in this split finding method is that it only visits the non-missing values[48]. The presented algorithm treats the non-presence as a missing value and learns the best direction to handle missing values. It starts by assigning a default direction (direction on tree, left or right) out of two possible directions and then during the training process it learns which is the best default direction for treating missing values.

Algorithm 3: Sparsity-aware Split Finding

Input: I , instance set of current node
Input: $I_k = \{i \in I | x_{ik} \neq \text{missing}\}$
Input: d , feature dimension
Also applies to the approximate setting, only collect statistics of non-missing entries into buckets
 $gain \leftarrow 0$
 $G \leftarrow \sum_{i \in I} g_i, H \leftarrow \sum_{i \in I} h_i$
for $k = 1$ **to** m **do**
 // enumerate missing value goto right
 $G_L \leftarrow 0, H_L \leftarrow 0$
 for j **in** $\text{sorted}(I_k, \text{ascent order by } \mathbf{x}_{jk})$ **do**
 $G_L \leftarrow G_L + g_j, H_L \leftarrow H_L + h_j$
 $G_R \leftarrow G - G_L, H_R \leftarrow H - H_L$
 $score \leftarrow \max(score, \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda})$
 end
 // enumerate missing value goto left
 $G_R \leftarrow 0, H_R \leftarrow 0$
 for j **in** $\text{sorted}(I_k, \text{descent order by } \mathbf{x}_{jk})$ **do**
 $G_R \leftarrow G_R + g_j, H_R \leftarrow H_R + h_j$
 $G_L \leftarrow G - G_R, H_L \leftarrow H - H_R$
 $score \leftarrow \max(score, \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda})$
 end
end
Output: Split and default directions with max gain

Figure 3.12. Sparsity-aware algorithm for split finding[48].

Light Gradient Boosting Machine (LightGBM)

LightGBM, same as xGBT, is an implementation of GBT. So mathematically speaking in terms of learning they are almost the same. LightGBM also achieves state of the art results in many tasks. The difference between LightGBM and other GBT variations is again on the splitting algorithm. It introduces two novel algorithms called Gradient-based One Side Sampling (GOSS) and Exclusive Feature Bundling (EFB)[23].

GOSS focuses on gradients of data instances. It is based on the principle that data instances with low gradient does not give important information[23]. That is, if an instance is associated with a small gradient, the training error for this instance is small and it is already well-trained. In order to not drop the instances with small gradients (it affects the data distribution and model accuracy) GOSS addresses this problem with a very simple approach, it keeps all the data instances with high gradients and random samples the data instances with low gradients(Figure 3.13). GOSS firstly sorts the data instances according to the absolute value of their gradients and selects the top $a * 100\%$ instances. Then it randomly samples $b * 100\%$ instances from the rest of the data. Then to compensate for the influence of the data, GOSS multiplies the instances with small gradients by a constant factor $1-a/b$. So it tries to give more importance to high gradient instances in order to maximize the information gain.

Algorithm 2: Gradient-based One-Side Sampling

Input: I : training data, d : iterations
Input: a : sampling ratio of large gradient data
Input: b : sampling ratio of small gradient data
Input: $loss$: loss function, L : weak learner
 $models \leftarrow \{\}$, $fact \leftarrow \frac{1-a}{b}$
 $topN \leftarrow a \times len(I)$, $randN \leftarrow b \times len(I)$
for $i = 1$ **to** d **do**
 $preds \leftarrow models.predict(I)$
 $g \leftarrow loss(I, preds)$, $w \leftarrow \{1, 1, \dots\}$
 $sorted \leftarrow GetSortedIndices(abs(g))$
 $topSet \leftarrow sorted[1:topN]$
 $randSet \leftarrow RandomPick(sorted[topN:len(I)], randN)$
 $usedSet \leftarrow topSet + randSet$
 $w[randSet] \times = fact$ ▷ Assign weight $fact$ to the small gradient data.
 $newModel \leftarrow L(I[usedSet], -g[usedSet], w[usedSet])$
 $models.append(newModel)$

Figure 3.13. Gradient-based One Side Sampling algorithm[23].

The second improvement of LightGBM has more to do with efficiency but is

important because it tries to improve efficiency by exploiting data sparsity presented by the data sample. The reason why sparsity plays a huge role in this technique is because sparsity gives us a lot of mutually exclusive features that can be bundled in a lossless way. The approach used is called Exclusive Feature Bundling[23]. The general idea behind it is that it creates feature bundles out of exclusive features. This is achieved in two steps by two different algorithms, the first algorithm in Figure(3.14) choses which features to bundle and the second algorithm deals with the way the features are bundled. This algorithm makes possible that the training time is not anymore dependent on the number of features but is dependent on the number of bundles, and since the number of bundles tends to be much smaller than the number of features, the training time can be reduced by a considerable amount of time.

Algorithm 3: Greedy Bundling

Input: F : features, K : max conflict count
Construct graph G
 $\text{searchOrder} \leftarrow G.\text{sortByDegree}()$
 $\text{bundles} \leftarrow \{\}, \text{bundlesConflict} \leftarrow \{\}$
for i **in** searchOrder **do**
 $\text{needNew} \leftarrow \text{True}$
 for $j = 1$ **to** $\text{len}(\text{bundles})$ **do**
 $\text{cnt} \leftarrow \text{ConflictCnt}(\text{bundles}[j], F[i])$
 if $\text{cnt} + \text{bundlesConflict}[i] \leq K$ **then**
 $\text{bundles}[j].\text{add}(F[i])$, $\text{needNew} \leftarrow \text{False}$
 break
 if needNew **then**
 Add $F[i]$ as a new bundle to bundles
Output: bundles

Algorithm 4: Merge Exclusive Features

Input: numData : number of data
Input: F : One bundle of exclusive features
 $\text{binRanges} \leftarrow \{0\}$, $\text{totalBin} \leftarrow 0$
for f **in** F **do**
 $\text{totalBin} += f.\text{numBin}$
 $\text{binRanges}.\text{append}(\text{totalBin})$
 $\text{newBin} \leftarrow \text{new Bin}(\text{numData})$
for $i = 1$ **to** numData **do**
 $\text{newBin}[i] \leftarrow 0$
 for $j = 1$ **to** $\text{len}(F)$ **do**
 if $F[j].\text{bin}[i] \neq 0$ **then**
 $\text{newBin}[i] \leftarrow F[j].\text{bin}[i] + \text{binRanges}[j]$
Output: newBin , binRanges

Figure 3.14. Exclusive Feature Bundling algorithms, algorithm number 3 selects the features to bundle and algorithm number 4 does the merging part[23].

3.6 Conclusions

In this chapter some basic and some advanced architectures for machine learning were presented together with some of the most common learning paradigms. Starting from a brief introduction for ML with the aim of showing what is the essence and what are some of the learning paradigms that are used from different algorithms as approaches to learn from the data like supervised learning, unsupervised learning, reinforcement learning, etc.

Once the premises for understanding how machine learning works some basic machine learning algorithms were introduced where the most commonly known examples are linear regression and decision trees which are the building blocks behind the advanced algorithms used in this work. Moreover a more complex type of learning was presented, called ensemble learning that is responsible for combining different learning algorithms (usually weak learners) into a more accurate

and powerful algorithm, this explains exactly the type of learning that is used in this work.

After all the needed information was presented, some more advanced machine learning algorithms were introduced. Starting with Multilayer perceptron that tries to mimic how humans learn and continuing with algorithms like Gradient Boosting Tree, Extreme Gradient Boosting Tree, etc. that are actually an ensemble of decision trees that can be used for classification or regression and incorporates different advanced algorithms for splitting and sampling the data.

Chapter 4

System Architecture

This chapter is structured as follows. In Section 4.1, the novel approach that was developed in this work, with details on both the architecture and its functions at a high level, are introduced. In Section 4.2, a more detailed explanation on some of the preprocessing approaches done in this work. In Section 4.3, the developed system architecture is presented. Finally, in Section 4.4, considerations on the novel implemented architecture are summarized.

4.1 Method

The novel proposed method leverages all the selected features, through the usage of a joint architecture based on tree ensemble models such as Catboost, GradientBoosting, Extreme Gradient Boosting, Light Gradient boosting and some simple models like ridge regression, kernel ridge and elastic net. Said features are the result of a preprocessing based on the techniques explained in Chapter 2 which will be explained in the next section in detail.

The proposed system can be splitted in two levels. The first level composed of powerful learners, takes as input the features that are a matrix of size $m \times n$ and produces a matrix P_i of size $1 \times n$ for each learner and since we have multiple learners in this level the overall output is set of $1 \times n$ matrices $P = p_1, p_2, p_3, \dots, p_d$ and has a dimension $d \times n$ where d is the number of learners that compose this level of the model.

The second level can also be called the meta learner, it consists in a simple supervised model. In this level the used models were ridge regression and kernel ridge since we have two subtasks. At this point the model takes as input the produced matrix P from the first level and produces a $1 \times n$ matrix that represents the final prediction for each of the subtasks.

The overall architecture can be seen as a stacked ensemble where the learners

of the first level are trained on the same data but they came out with different properties so that is why we use a second level that tries to learn how to use these properties to create a more powerful and robust model. The following schema illustrates this ensemble.

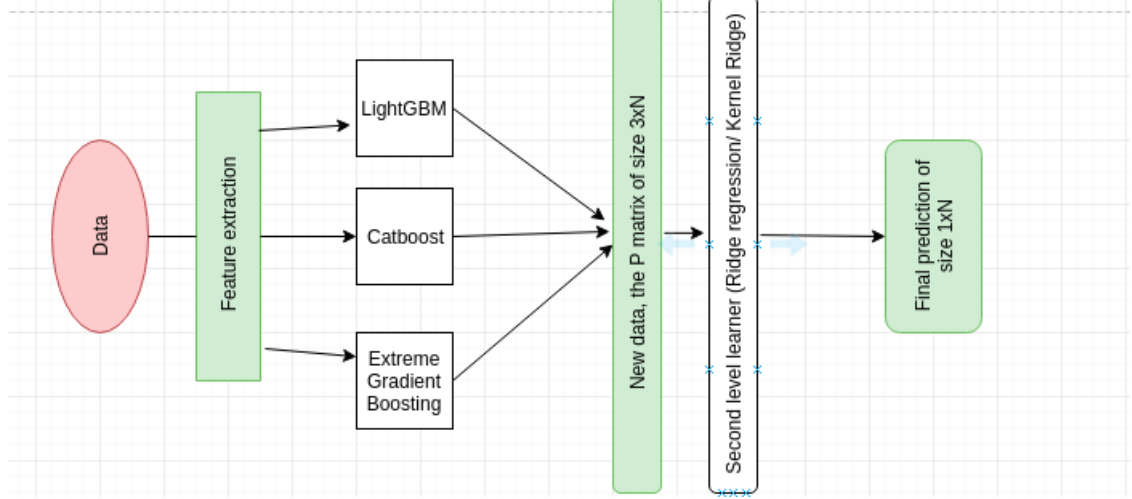


Figure 4.1. Proposed method architecture. Image generated from Diagram Editor <https://www.diagrameditor.com>

4.2 Feature Preprocessing

The features used to learn from the algorithms are of different nature, like continuous features, ordinal features, categorical features, etc. Since the data are semi structured, it comes with a lot of problems and is very crucial to process them in the right way. During preprocessing the four main problems to face were high cardinality, outliers, scaling and feature selection

Dealing with high cardinality features: High cardinality means that the number of categories inside one variable is too high to be covered from the number of samples[36]. In this work, this problem was faced with a very important feature that represents the main diagnosis of rehabilitation. The classification was done based on the Italian Classification of Disease which was not appropriate for a dataset of that size. The solution we came up with was to replace this kind of classification with ICD-9 (International Classification of Diseases, Ninth Revision) which is the official standard for assigning codes to diagnoses and procedures associated with hospitals. This way we could have a reliable classification and appropriate to work with (this decision was made after consulting experts of the field).

Dealing with outliers: Outliers are values that should be analysed very carefully.

Dropping them might bring the loss of something important and sometimes we lose generalization, also keeping them may bring confusion to the model. We had to face this issue when dealing with patient skills and abilities level before rehabilitation and after. The dataset was rich in outliers that represent different situations and some of them were necessary to be dropped. An example of this was when we had patients with skills and abilities level after rehabilitation lower than the level before rehabilitation. These are extreme cases where rehabilitation has no effect, very low in number and are not of interest so a good decision is to drop them.

Dealing with scaling and normalization: Continuous features in the dataset were on different scales and also the distributions were not normal. Is not a must to do scaling and normalization but it can help. With scaling we make sure that the algorithms are not falsly affected by variables which have a high scale more than variables of a lower scale[13]. Normalization on the other hand helps to extract more important statistics. In this work we used box-cox transformation in some of the variables where it has performance improving impacts. One of the variables we used was "Giorni di degenza" which directly shows the efficiency of rehabilitation. Box-cox is actually an exponential transformation based on the following formula:

$$y(\lambda) = \begin{cases} \frac{y^\lambda - 1}{\lambda} & \lambda \neq 0 \\ \log_y & \lambda = 0 \end{cases} \quad (4.1)$$

The procedure is to find the right lambda that gives the best approximation with a normal distribution to our data. The outcome was the following:

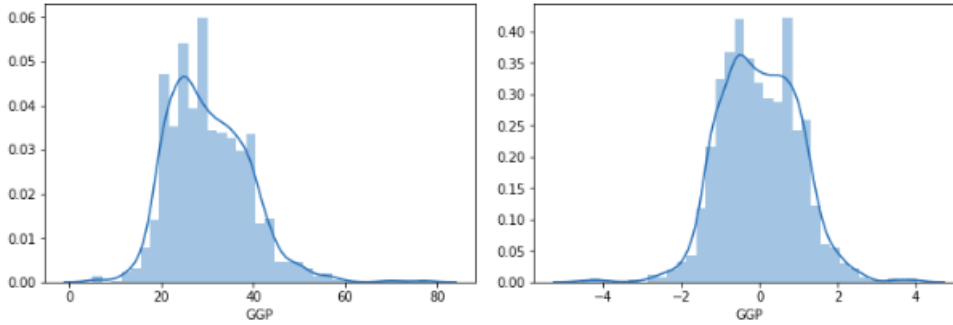


Figure 4.2. GGP distribution before and after box-cox transformation. Images generated from seaborn library

Other feature transformations: Transformations on features done on this work were many. We dealt with many kinds of variables and each of them had the appropriate transformation. An example of it is to increase the scale (reverse of scaling) of important features when you want them to have a bigger impact on the learning algorithm. We used squaring on some of the variables to increase their scale and impact which resulted in better performance.

Dealing with feature selection: Since the dataset had many features it was important to select which features to use for learning. To select the feature a wrapper method was used where different subsets of features were selected randomly and the one with the best performance was selected (greedy algorithm) as the final feature subset[19]. The number of features was reduced from 127 to 79 features.

4.3 Model Architecture

As mentioned in the previous method section, the proposed architecture or model is composed of two different levels: an upper level, based on ensemble tree models like lightGBM, xGBOOST, etc. operating on the preprocessed features and a lower level called differently the meta learner level based on a simple regression model, operating on the features produced from the upper level learners. The overall architecture represents an ensembling algorithm called stacked ensemble. The predictions made by the upper level learners, as mentioned above, are stacked together and represent the learned features from the data. Meanwhile the output of the lower level learner represents the final output.

The tree ensemble methods were the models of choice due to remarkable results obtained in regression tasks during the years. A quick review of the mathematical equations of these models taken from chapter 3 follows. For a given data set with n examples and m features $D = (x_i, y_i) (|D| = n, x_i \in R_m, y_i \in R)$, a tree ensemble model uses K additive functions to predict the output[38]. The prediction formula is:

$$\hat{y}_j = \phi(\mathbf{x}_i) = \sum_{k=1}^K f_k(\mathbf{x}_i), f_k \in F \quad (4.2)$$

where $F = f(x) = wq(x)$ ($q : R_m \Rightarrow T, w \in R, T$). Q represents the structure of each tree that maps an example to the corresponding leaf index and each f_k corresponds to an independent tree structure q and leaf weight w . T is the number of leaves in the tree. All the other models we have used have almost the same mathematical function used for prediction they only differ in the splitting algorithms they use. So based on this formula the first predictions are produced directly from the features creating a $n \times m$ data matrix where n ($n > 0$) represents the number of samples and m ($m > 1$) represents the number of learners.

Regarding the lower level, it is based on a simple ridge regression that takes as input the produced $n \times m$ matrix and produces an output of the shape $n \times 1$ based on the following formula:

$$L = \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda \sum_{i=1}^n \beta_i^2 \quad (4.3)$$

Where $\hat{Y}_i$ is the estimated value, Y_i is the ground truth and β_i^2 is the penalisation term. So basically ridge regression is an RSS plus a squared penalisation term.

4.4 Conclusions

In this Chapter a detailed explanation of the novel devised method was presented. The implemented architecture was described in depth, illustrating how different learners were combined together and how the features were feeded to the model. A specific pipeline, where the two levels of the architecture takes the inputs and produces the prediction in two steps, with the first step consisting in collecting outputs of different learners and the second step which combines those outputs through a simple learner. After this pipeline description, information on the utilised features was also provided, describing how features were generated and fit into the devised system. Finally, a review for the equations of the implemented components was presented.

Chapter 5

Test and evaluation

This chapter is organized as follows. In Section 5.1, the dataset containing the data about patients necessary for predicting effectiveness and efficiency is introduced. In Section 5.2, all the information regarding the machine utilised to run the system as well as details on the used models, are highlighted. In Section 5.3 a detailed comparison with SOTA works is shown. Finally in Section 5.4 a review of the shown results is highlighted.

5.1 Dataset

The dataset used for the training phase as well as a benchmark for the experiments, is a dataset provided from the hospital "Policlinico Umberto I". In this collection, data about the patients rehabilitated in orthopedic or neurology from 2015 to 2018, are retained. Regarding the number of different patients recorded during this period, 4050 individuals were monitored during their rehabilitation period. For the stored data, the chosen format is the CSV, which provides a tabular format that is easy to work with and to mine for finding relevant information.

The data is very rich in features and contains information that can be useful for two different tasks but very similar, effectiveness and efficiency of rehabilitation all maintaining the format specified in "Scheda RAD-R". The features or the data accumulated for each patient are a mix of quantitative variables (nearly 10% of the features) and qualitative variables (nearly 90%) that contain personal information about the patient and about their medical record. The two variables that served as targets (each of them for different tasks) are "Barthel in uscita" and "Giorni di degenza". Barthel in uscita represents the level of skill the patient has after the rehabilitation measured based on the protocol described on "Scheda RAD-R" and "Giorni di degenza" represents the amount of days the patient needed to recover which is directly related with rehabilitation efficiency.

In order to select meaningful data to use in supervised learning algorithms like

the one used in this work, it was very important to make a detailed analysis of the data in order to find the most relevant features. After a deep analysis, for the quantitative features, it came out that age was very important for determining the potential of improvement and also for the time needed to recover from the intervention. In terms of relevance for the tasks we can say the same for "Barthel ingreso" which shows how skillful the patient was when entering the hospital and starting the rehabilitation.

On the other hand the qualitative variables were many in numbers but some of them stand out in terms of relevance for the purpose of this study. First, the pathology subject to the intervention which in medical records is known as cod_26, among all qualitative variables it is the most relevant. Secondly, the base pathology related to the intervention which is known as cod_27. Thirdly, a group of variables (equally relevant) that shows diseases associated with the patient during the rehabilitation period which gives a strong indicator about the conditions of the immunitary system of the patients. During the work, after a detailed analysis of the data and consultation with experts of the field the values of these qualitative variables were replaced with new values changing the classification system of disease into a more appropriate one which is widely accepted by hospitals and is known as ICD-9 (International Classification of Diseases, Ninth Revision).

5.2 Implementation details and model analysis

In order to evaluate the proposed system, an extensive experimental phase has been performed. The experiments were executed on a 4-Core Intel i5 2.20Hz 8th generation CPU with 16GB RAM and a GPU M110 of NVIDIA. The libraries used to implement the proposed system were pandas[51], numpy[52], scikit learn [47], catboost[48], xGBoost[49] and lightGBM[50]. For comparing the system with previous state of the art works, a 10-cross validation method was applied.

Regarding the used architectures, there are four different ones, each for a specific task and for specific rehabilitation departments (neurology and orthopedic). For rehabilitation effectivity in neurology, the used architecture consists in a first layer composed of Catboos (from catboost library), gradient boosting (scikit learn) and xgboost (xgboost library) and a second layer or meta layer which is a simple KernelRidge regression (scikit learn). For rehabilitation effectivity in orthopedic, the used architecture consists of a first layer composed of Catboos, lightGBM (lightGBM library) and xgboost and a second layer or meta layer which is a simple Ridge regression (scikit learn). On the other hand for rehabilitation efficiency in neurology, the used architecture consists of a first layer composed of gradient boosting, ElasticNet (scikit learn) and Lasso regression (scikit learn) and a second layer which is a simple Ridge regression. Finally the used architecture for rehabilitation efficiency in orthopedic has a first layer composed of lightGBM, ElasticNet and gradient boosting and a second level which consist in a KernelRidge regression.

The reason for this different configurations where the accuracies they would provide for each specific case. It is important to note that all the models where hyperparameter tuned[55] using grid search algorithms[54] in order to have the best outcome possible in the given conditions. The result from testing different architectures on all possible cases are given in the following tables where the first table gives the results for effectivity meanwhile the second one for efficiency.

Model	Error in Orthopedy Effectiveness	Error in Neurology Effectiveness
xGBT+gradient boosting+catboost	6.71	8.5
lightGBM + catboost + xgboost	6.54	8.9
Gradient boosting + elastic net + lasso	6.8	9.23
Gradient boosting + elastic net + lightGBM	6.58	9.09

Figure 5.1. Experimental comparison between different models on Effectiveness.

Model	Error in Orthopedy Efficiency	Error in Neurology Efficiency
xGBT+gradient boosting+catboost	8.07	10.66
lightGBM + catboost + xgboost	8.07	10.55
Gradient boosting + elastic net + lasso	7.99	10.66
Gradient boosting + elastic net + lightGBM	8.03	10.44

Figure 5.2. Experimental comparison between different models on Efficiency.

5.3 Comparison

A good evaluation phase also needs to compare the obtained results against other state-of-the-art approaches. For this reason, the proposed method was also compared to the current literature in rehabilitation efficiency and effectiveness. The comparison is done in two parts, first a comparison with a previous work on the same task and then a model comparison on the same dataset with models that achieve state of the art on similar machine learning tasks. The results on the only available dataset, shown in the following tables (fig. 5.3 and 5.4), highlights how our method exceeds the pioneer work of Seccia et al. et al. [56]. In both tasks the proposed model outperforms the models of Seccia et al. et al providing a better accuracy for

efficiency and effectiveness also.

System	Efficiency Orthopedy	Effectiveness Orthopedy
Seccia et al. [56]	9.32	7.65
Proposed system	7.99	6.54

Figure 5.3. Experimental comparison on the given dataset for Orthopedy.

System	Efficiency Neurology	Effectiveness Neurology
Seccia et al. [56]	11.2	10.73
Proposed system	10.44	8.5

Figure 5.4. Experimental comparison on the given dataset for Neurology.

Finally, in the following tables (fig. 5.5 and 5.6) the comparison between our model and other state-of-the-art models is done. In our model we have used some ensemble algorithms like catboost, xGBoost and lightGBM that are actually models that on their own produce SOTA results in a lot of tasks in machine learning, results that are compared with those of neural networks. So the scope of this comparison is to understand what was the added value of the novel proposed model compared to the original algorithms. In the two following tables is shown the overall result and how our proposed models outperforms the original implementations in all the tasks.

System	Error in Orthopedy Effectivity	Error in Neurology Effectivity
xGBT[49]	6.6	8.65
lightGBM [50]	6.56	8.55
Catboost[48]	6.71	8.7
Our model	6.54	8.5

Figure 5.5. Experimental comparison on the given dataset with SOTA models on Effectiveness

System	Error in Orthopedy Efficiency	Error in Neurology Efficiency
xGBT[49]	8.2	11.02
lightGBM [50]	8.5	10.6
Catboost[48]	8.05	10.52
Our model	7.99	10.44

Figure 5.6. Experimental comparison on the given dataset with SOTA models on Efficiency

5.4 Conclusions

In this Chapter the extensive performed tests were reported. Initially, a brief presentation of the used dataset was made emphasizing the usability of it in two different tasks, effectiveness and efficiency. A brief description of the machine used to perform training and testing was also introduced as well as the different architectures and combination of architectures used in this work for the two tasks and how the hyperparameters were chosen for each model.

Since a modular architecture was designed to solve the tasks, various configurations were tested to find the right components that could support these tasks. In more details, all the models that are part of the ensemble architecture were tested separately on the tasks due to the fact that they have proven to give SOTA results on a huge number of tasks in machine learning especially in semi structured data like in this case. Testing and comparing the proposed model against these architectures justifies the reason behind stacking these models since it introduces performance gain. Finally, a comparison between this work and a previous SOTA work was

done by comparing the average 10-fold cross validation results. As was shown, the obtained score highlights how the devices method outperforms the other models in predicting effectiveness and efficiency.

Chapter 6

Conclusions

In this work, an original combination of machine learning models and careful data preprocessing are used in order to realize an original method for predicting effectiveness and effectivity in rehabilitation. An exhaustive experimental phase was performed on the dataset available in the current literature provided from the hospital "Policlinico Umberto I". The obtained results showed how the proposed solution outperforms key works in this topic, thus demonstrating how the combination of models can help in improving the accuracy of prediction for effectiveness and efficiency. As for future development, interesting topics concern the investigation of new models, deep learning models and also other feature engineering approaches that could help in extracting more useful information. To conclude, an improvement of the dataset is another relevant step that would be carried out by integrating new samples because the dataset is very small and to improve the quality of data, because of the human factor a lot of samples are contradictory with each other and makes it very difficult for the models to learn.

Bibliography

- [1] Clifford, J. *The UN disability convention and its impact on European equality law*. The Equal Rights Review 6 (2011) (2020).
- [2] Patsy Chow, MPH and Cynthia Chen BSc *Factors and Trade-Offs With Rehabilitation Effectiveness and Efficiency in Newly Disabled Older Persons*, 2014
- [3] Prateek Grover, MD and Denise Holt, *Outcome Measurement in Rehabilitation*, 2014
- [4] FI Mahoney, *Functional evaluation: the Barthel Index: a simple index of independence useful in scoring improvement in the rehabilitation of the chronically ill*, 1965
- [5] Lei Lu, Ying Tan, *Evaluating rehabilitation progress using motion features identified by machine learning*, 2020.
- [6] Yaar Harari, Megan K. O'Brien, *Inpatient stroke rehabilitation: prediction of clinical outcomes using a machine-learning approach*, 2020
- [7] Dr. Patrick Philipp, *Continuous support for rehabilitation using machine learning*, 2019
- [8] Mu Zhu¹, Zhanyang Zhang¹, *Using machine learning algorithms to guide rehabilitation*, 2018.
- [9] Bishop, C. M. (2006), *Pattern Recognition and Machine Learning*, Springer, ISBN 978-0-387-31073-2
- [10] Pyle, D., *Data Preparation for Data Mining*. Morgan Kaufmann Publishers, Los Altos, California., 1999
- [11] Wu, S., *A review on coarse warranty data and analysis" (PDF)*, *Reliability Engineering and System*, 2013.
- [12] *Machine Learning and AI via Brain simulations*. Stanford University.
- [13] Hussain Alkharusi, *Variables in Regression Analysis: A Comparison of Dummy and Effect Coding*, 2012
- [14] Roweis, S. T.; Saul, L. K. *"Nonlinear Dimensionality Reduction by Locally Linear Embedding*, 2000

- [15] Jenkins, O.C. and Mataric, M.J. *Deriving Action and Behavior Primitives from Human Motion Data. International Conference on Robots and Systems*, 2002.
- [16] A. G. K. Janecek and G. F. Gansterer et al, *On the Relationship between Feature Selection and Classification Accuracy, In: Proceeding of New Challenges for Feature Selection*, pp. 40-105, 2008.
- [17] Hamon, Julie. *Optimisation combinatoire pour la selection de variables en regression en grande dimension : Application en genetique animale (Thesis) (in French). Lille University of Science and Technology*, 2013.
- [18] Yu, Lei; Liu, Huan. *Feature selection for high-dimensional data: a fast correlation-based filter solution* . ICML'03: Proceedings of the Twentieth International Conference on International Conference on Machine Learning: 856-863.
- [19] Shamila Nasreen *A Survey Of Feature Selection And Feature Extraction Techniques In Machine Learning*, SAI, 2014
- [20] Saghapour, E.; Kermani, S.; Sehhati. *A novel feature ranking method for prediction of cancer stages using proteomics data*, 2017.
- [21] N. Chumerin and V. Hulle, M. M, *Comparison of Two Feature Extraction Methods Based on Maximization of Mutual Information* In: Proceedings of the 16th IEEE Signal Processing Society Workshop on Machine Learning for Signal Processing, pp. 343-348, 2006.
- [22] Grus, Joel *Data Science from Scratch*. Sebastopol, CA: O'Reilly. pp. 99, 100. ISBN 978-1-491-90142-7
- [23] *LightGBM: a highly-efficient gradient boosting decision tree*. Neural Information Processing Systems (NIPS). Retrieved 2019-12-18.
- [24] E. Rosten and T. Drummond, *Machine learning for high-speed corner detection*, in European conference on computer vision, pp. 430-443, Springer, 2006.
- [25] G. Bradski and A. Kaehler, *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media, Inc.", 2008.
- [26] Wahab Khan, *A survey on the state-of-the-art machine learning models in the context of NLP*, 2016
- [27] Hinton, Geoffrey; Sejnowski, Terrence . *Unsupervised Learning: Foundations of Neural Computation* MIT Press, 1999.
- [28] [28] Csaba Szepesvári, *Algorithms for Reinforcement Learning Synthesis Lectures on Artificial Intelligence and Machine Learning*, 2010.
- [29] van Otterlo, M.; Wiering, M. *Reinforcement learning and markov decision processes. Reinforcement Learning. Adaptation, Learning, and Optimization*. , 2012

- [30] Kaelbling, Leslie P.; Littman, Michael L.; Moore, Andrew W. (1996). *Reinforcement Learning: A Survey*. Journal of Artificial Intelligence Research. 4: 237–285.
- [31] David A. Freedman (2009). *Statistical Models: Theory and Practice*. Cambridge University Press. p. 26.
- [32] Lehmann, E. L., Casella, George (1998). *Theory of Point Estimation (2nd ed.)*. New York: Springer.
- [33] Bühlmann, Peter; Van De Geer, Sara (2011). *Statistics for High-Dimensional Data*. Springer Series in Statistics: 9
- [34] Santosa, Fadil; Symes, William W. (1986). *Linear inversion of band-limited reflection seismograms*. SIAM Journal on Scientific and Statistical Computing.
- [35] Kennedy, Peter (2003). *A Guide to Econometrics (Fifth ed.)*. Cambridge: The MIT Press.
- [36] Zou, Hui; Hastie, Trevor (2005). *Regularization and Variable Selection via the Elastic Net*. Journal of the Royal Statistical Society, Series B: 301–320.
- [37] Rokach, Lior; Maimon, O. (2008). *Data mining with decision trees: theory and applications*
- [38] Shalev-Shwartz, Shai; Ben-David, Shai (2014). *18. Decision Trees". Understanding Machine Learning*.
- [39] Rokach, L.; Maimon, O. (2005). *Top-down induction of decision trees classifiers-a survey*. IEEE Transactions on Systems, Man, and Cybernetics - Part C: Applications and Reviews. 35
- [40] Freund, Y.; Schapire, R. E. (1999). *Large margin classification using the perceptron algorithm*. Machine Learning. 37
- [41] Opitz, D.; Maclin, R. (1999). *Popular ensemble methods: An empirical study*. Journal of Artificial Intelligence Research. 1
- [42] Polikar, R. (2006). *Ensemble based systems in decision making*. IEEE Circuits and Systems Magazine.
- [43] Rokach, L. (2010). *Ensemble-based classifiers*. Artificial Intelligence Review. 33 (1–2): 1–39.
- [44] Patricio Cerda, Gael Varoquaux (2020) *Encoding high-cardinality string categorical variables*
- [45] Angelos Chatzimpampas, Rafael M. Martins, Kostiantyn Kucher, and Andreas Kerren, StackGenVis: *Alignment of Data, Algorithms, and Models for Stacking Ensemble Learning Using Performance Metrics.*, 2020
- [46] Cheng Li. *A Gentle Introduction to Gradient Boosting*, 2008

- [47] Fabian Pedregosa, *Scikit-learn: Machine Learning in Python.*, 2011
- [48] Anna Veronika Dorongush, *CatBoost: gradient boosting with categorical features support*, 2017
- [49] Shubham Malik, *XGBoost: A Deep Dive into Boosting (Introduction Documentation)*, 2020
- [50] Huan Zhang, *GPU-acceleration for Large-scale Tree Boosting*, 2017
- [51] Wes Mckinney, *pandas: a Foundational Python Library for Data Analysis and Statistics*, 2011
- [52] Stefan Van Der Walt, *The NumPy array: a structure for efficient numerical computation*, 2011
- [53] Sebastian Raschka, *Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning*, 2018
- [54] Petro Liashchynskyi, *Grid Search, Random Search, Genetic Algorithm: A Big Comparison for NAS*, 2019
- [55] James Bergstra, *Algorithms for Hyper-Parameter Optimization*, 2011
- [56] Jian Jiang, Rui Wang, *Boosting Tree-Assisted Multitask Deep Learning for Small Scientific Datasets*, 2020
- [57] Ruggiero Secciaa, *Data of patients undergoing rehabilitation programs*, 2020.
- [58] Panagiotis G. Asteris, *Feed-Forward Neural Network Prediction of the Mechanical Properties of Sandcrete Materials*, 2017